THE PROBLEM · RECOVER AND GROW WITH AI

Cart recovery today buys back orders, not profit

Abandoned-checkout tools are set up and judged on orders recovered. The discount goes to every shopper who comes back, including those who would have bought anyway, so recovery can quietly erode the merchant's margin.

Why this is a Track 2 problem

- **Recover:** abandoned carts are a large, recurring revenue leak. Baymard's average cart abandonment is about 70%.
- **Grow:** the prize isn't more orders. It's more **margin kept** on the orders recovered.
- **AI-native:** the right discount depends on who the shopper is, how they pay, when they shop and what's in the cart. No fixed rule can capture that.

Merchant: a D2C brand on Magic Checkout

- Several categories with very different margins, e.g. serums 72% vs protein 42%
- Sends a flat or tiered coupon to abandoned carts and reads success as recovered orders
- Has no way to see which discounts created a sale and which gave margin away

How recovery works today

Shopper leaves checkout
Before paying, or after a failed UPI, card or OTP attempt.

Abandonment event fires
Magic Checkout's abandoned-cart webhook, or an agent watching for it.

A preset offer goes out
Picked from the merchant's preset coupons (flat, tiered by cart value), never above the merchant's cap.

Success = orders recovered
No public tool measures what would have happened without the discount.

What Razorpay offers publicly (Oct 2026)

- **Magic Checkout** coupons are merchant-made: flat, %, tiered, BXGY, with cart-value conditions.
- **Agent Studio cart agent** picks among offers the merchant set up, never above the merchant's maximum.
- None of this public material describes a **control group, margin floor or incremental-profit report**.

The leak, in one table

Illustrative

1,000 abandoned carts, AOV ₹1,000, 40% margin. With a reminder alone, 150 shoppers come back and pay.

OFFER TO EVERY CART	ORDERS	PROFIT	VS REMINDER
Reminder only	150	₹60,000	—
5% off	170	₹59,500	−₹500
10% off	**200 (+33%)**	**₹60,000**	**₹0**
15% off	210	₹52,500	−₹7,500

Profit = orders × AOV × (margin − discount). At 10%, a third more orders earn **zero** extra profit: the 150 who'd have paid anyway now get ₹100 each.

```
Conversion dashboards report 10% as the winner. The
merchant's P&L doesn't.
```

Problem: recovery offers are chosen from a preset menu and judged on orders won back, so merchants pay discounts to shoppers who'd have returned anyway and can't see the margin they lose.
Opportunity: use the checkout signals Razorpay already sees to price each abandoned cart's offer for **profit kept** (often ₹0), and prove the gain against a reminder-only control group.

THE CHECKOUT JOURNEY · WHERE PROFIT IS MADE AND LOST

Maximise profit across both paths to purchase, not conversions

Every shopper who reaches checkout

Illustrative · N = 1,000 · AOV ₹1,000 · margin m = 40%

Reach checkout
N = 1,000
Magic Checkout opened; cart and contact known

α = 55% Pay straight through
550 orders
$Profit_1 = \alpha \cdot N \cdot AOV \cdot m = ₹2,20,000$
The agent never touches this path. No discount, no change.

1 − α = 45% Abandon
450 carts
Left before paying, or the payment failed. Each one gets a reminder (with consent).

c(d) Come back and pay
90–122 orders
$Profit_2 = (1-\alpha) \cdot N \cdot c(d) \cdot AOV \cdot (m - d)$
c(0) of them would return on the reminder alone. Any discount they get is **pure leak**.

1 − c(d) Drop off anyway
328–360 carts
No profit, whatever the discount. Shoppers who left over delivery dates, trust or just browsing don't come back for a bigger coupon.

Two objectives

Today maximise orders: $\alpha + (1-\alpha) \cdot c(d)$ → rises with every extra % of discount

This agent maximise profit kept

$$\alpha \cdot m \;+\; (1-\alpha) \cdot c(d) \cdot (m - d)$$

α·m
profit from shoppers who pay straight through. Offers only go out after abandonment, so **d can't change it**.

(1−α)·c(d)·(m − d)
abandoned share × share who come back at d × margin left after d. **This is the only part the agent controls.**

For each abandoned cart, the agent tries every d from 0 to the cap and keeps the highest **c(d | x)·(m − d)**. A bigger d raises c but shrinks (m − d), and the best point is often **₹0**.

The 450 abandoned carts under each policy

POLICY	C(D)	ORDERS	$PROFIT_2$	VS REMINDER
Reminder only	20%	90	₹36,000	—
Flat 10% to all	27%	122	₹36,600	+₹600
Agent (targeted, avg ₹36 an order)	26%	117	**₹42,600**	**+₹6,600**

5 fewer orders than flat 10%, but **11× the extra profit**.

Notation

When does a discount pay? Only if **(c(d) − c(0))·(m − d) > c(0)·d**: extra orders × margin left must beat the discount handed to shoppers who'd have returned anyway. Flat 10% here: 2.1 vs 2.0, so it barely pays.

α checkout conversion rate (pay without abandoning) · **c(d)** recovery conversion rate at discount d; **c(0)** with a reminder only · **m** gross margin, as a share of selling price · **d** discount, as a share of eligible items' price · **x** the cart's signals.

THE SOLUTION · AN AGENT STUDIO AGENT

Margin-Aware Cart Recovery prices every abandoned cart's offer for profit

STEP 1 — Trigger
Abandoned checkout or failed payment (Magic Checkout event + gateway result)

→

STEP 2 — Scope
Cart has ≥1 of the merchant's top-5 margin categories; otherwise the normal flow runs

→

STEP 3 — Read signals
10 checkout signals plus the categories in the cart

→

STEP 4 — Predict
Chance of purchase c(d | x) at every discount from 0 to the cap, in 0.5% steps

→

STEP 5 — Decide
Pick d* for maximum expected profit within cap, floor and blocked groups; or assign to test or control

→

STEP 6 — Message
"Save ₹75" on eligible items only. Failed payment → retry link first

→

STEP 7 — Learn
Outcome logged, model re-fit, control group measures the lift

Signals the AI reads

Bucket-level · no shopper profiles

Shopper Customer-level · switchable
First-time · repeat

Payment status
Left before paying · bank decline · UPI timeout · OTP failed

Day
Weekday · weekend · payday week (1st–7th)

Region Switchable
Metro · tier 1 · tier 2 · tier 3, from pincode, tier only

Coupon behaviour
Didn't look · opened coupons · tried a code that failed

Categories in cart Always on
Each of the five categories as a yes/no input; margin per item

Payment method
UPI · credit · debit · net banking · COD · not chosen

Time of day
Six slots, from early morning to late night

Season
Regular · sale event · festive

Cart size
Relative to the merchant's own AOV: <0.5× · 0.5–1× · 1–2× · >2×

Traffic source
UTM: organic · paid social · search · influencer · email/WhatsApp

Device · income bracket Excluded
Never used: customer-level personal data and a proxy for spending power; DPDP and fairness risk

Why Magic Checkout is the trigger: it is Razorpay's own D2C checkout, so at drop-off it already holds the cart, the payment attempt and its failure reason, the consented contact and the UTM source, and it emits an abandoned-cart event. No new tracking is needed. **Why first-time vs repeat:** repeat buyers already trust the brand and convert about 2× more on a reminder alone, so a discount mostly leaks margin there.

What the merchant sets

- **Top 5 categories by margin.** Only carts containing one are in scope, and only those items are discounted. Low-margin items have no room to discount.
- For each category: a **product type** (shared taxonomy), a **margin %** (private, never pooled) and a **max discount %**.
- A **minimum margin after discount**, the channel (WhatsApp or email), and review before going live.
- Entered as a form or **uploaded as Excel or PDF**. The AI extracts the values and shows a diff to confirm.

How it learns from outcomes

- **10% control:** reminder only, always on. This is the yardstick for every profit claim.
- **Random test offers** from 0 to the cap in 0.5% steps. Randomised offers are what make the discount effect causal, not just correlated.
- **Explore → learn → optimise:** 100% → 45% → 12% of carts keep testing (a bandit in production).
- **New merchants don't start from zero:** Razorpay's existing checkout data, pooled by product type across live merchants, gives day-one curves, and the store's own carts refine them. Margins and shopper data are never pooled.
- Shoppers see a **rupee saving** rounded to ₹5, so odd percentages don't look arbitrary.

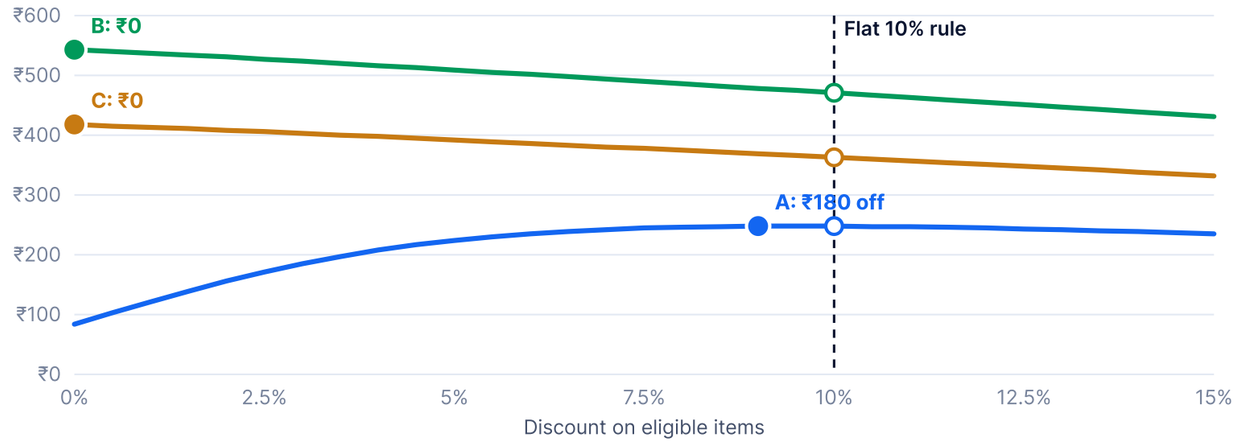
WHY AI, AND WHY NOT A FIXED RULE

One discount for everyone is right for almost no one

Expected profit per cart at each discount

From the prototype model · ₹1,998 serum + night cream cart

A — B — C ● agent's pick, the peak of each curve ○ what flat 10% gives y-axis: chance of purchase × margin left



A · First-time · COD · paid social · tier 2 · opened coupons
Best: **₹180 off** (9%) · flat 10% ≈ same

B · Repeat · UPI · organic · metro · afternoon
Best: **₹0** (reminder only) · flat 10% loses ₹72 a cart

C · UPI timeout (payment failed) · search · metro
Best: **₹0** + retry link · flat 10% loses ₹55 a cart

Signals are correlated: COD skews to Tier 2–3 and paid social, and repeat buyers skew prepaid. Group-by-group rules double count these overlaps. The model learns all signals together, so each one's effect on conversion is measured with the others held fixed. Randomised test offers keep the discount effect unbiased even when signals move together.

Extra profit per 1,000 abandoned carts vs reminder-only

Prototype simulation · 30 days

Glow & Co. · skincare		ThreadLine · apparel		FuelUp · sports nutrition	
Flat 10%	+₹37.5k	Flat 10%	+₹68.2k	Flat 10%	−₹15.1k
Tiered	+₹41.7k	Tiered	+₹70.2k	Tiered	−₹10.6k
Agent	+₹52.8k	Agent	+₹88.0k	Agent	+₹45.0k

Where the AI sits

Response model	ML	Predicts each cart's buy chance at every discount, from its signals
Optimiser	Math	Picks the profit-maximising discount within the merchant's fence
Pooling	ML	Shares discount response by product type across stores
Learning loop	ML	Random tests + control; explore → exploit (bandit)
LLM layer	GenAI	Maps products, reads margin files, explains and answers
Guardrails	Rules	Caps, floor, control, consent: deliberately **not** AI

Fixed rule vs the agent

	FLAT OR TIERED RULE	AGENT
Who gets paid	Every shopper who redeems it, including those who'd have returned on a reminder alone	Only carts where the discount changes the outcome; failed payments mostly ₹0
Inputs	One at most (cart value)	10 signals + category mix (~47,600 combinations no human could set)
Depth	One % for all	Each cart's own peak, where extra orders stop paying for the discount
Over time	Stale until someone notices	Re-learns through festive weeks, sales and channel shifts
Proof	Rarely tested	Always-on control measures the incremental profit

MERCHANT CONTROL, TRUST AND PROOF

The merchant sets the fence; the AI works inside it

Before → after for the merchant

	BEFORE	AFTER
Offer	Preset menu: flat or tiered by cart value	✓ Priced per cart from 10 signals, often ₹0
Success metric	Orders recovered	✓ Profit kept vs control
Failed payments	Same coupon	✓ Retry link first
Margin	Invisible to the tool	✓ Floor + cap per category
Proof	None	✓ 10% control, always on
Merchant effort	Tuning coupons by hand	✓ Approving a policy

Review · change · approve · stop

- **Review first:** any change to limits, signals or groups waits for **Approve and go live**.
- **Change:** caps, margin floor and categories, plus a switch per signal that shows its profit cost (region tier adds only about ₹0.4–1k per 1,000 carts).
- **Override:** set any group to **reminder only**, e.g. all COD carts.
- **Understand:** "what the agent learned" in plain words, a "Why?" per group, and the top drivers per cart.
- **Why the merchant keeps the upper limit:** pricing and the P&L are theirs, and they know what the model can't (stock, brand positioning, sale calendar). The cap bounds the worst case and makes the agent explainable. Agent Studio's own rule is that agents never exceed the merchant's maximum.
- **Stop:** pause the agent in one switch. Every cart keeps an audit log of offer, reason and checks.

Accuracy, failure and permissions

Too little data	Fall back to the merchant's own rule until a group has enough tests; confidence shown per group
Model drifts	Stop-loss pauses a group if its treated profit falls below control
Wrong margin	Margin floor applies; uploads show a diff before saving
Discount gaming	Offer cap per contact; random ₹0 offers keep abandoning unrewarding
Bad message	Pre-send checks: limits, floor, no pressure wording, opt-out line
Outage	Reminder only, never a bigger discount
Permissions	DPDP consent before any message · bucket-level signals · margins private · no list uploads

Business outcome measured

North star

Incremental profit per 1,000 abandoned carts = treated profit − control profit, after discounts

- **Efficiency:** discount ₹ per incremental order; share of offers at ₹0
- **Volume:** recovery rate vs control
- **Guardrails:** opt-out and complaint rate, margin-floor breaches (must be 0), returns on recovered orders
- **Adoption:** merchants who finish margin setup; share of recommendations approved

Assumptions → how to validate with real data

ASSUMPTION	TEST
Discount response differs by signal	4-week pilot with 3 merchants: random offers, compare lift by group
Merchants can't measure incrementality today	10 merchant interviews + Agent Studio usage data
Merchants will share margins	Setup completion rate in the pilot
Pooled curves transfer across stores	Compare pooled vs store-only predictions on held-out carts
Gaming stays small	Track repeat abandoners' offer rates

Deliberately left out

All categories	Low-margin items have no room to discount
Customer-level history	Privacy, and not needed for bucket-level pricing
Device and income signals	Customer-level personal data (DPDP) and a proxy for spending power
Raising or personalising list prices	MRP law and fairness; discounts only

Next Free shipping as a cheaper lever than a discount · list-price review from the merchant's own test evidence. **Open the live prototype →**